

Spike: Task 19**Title:** Message Systems**Author:** Mohamed Shafi Uzman Fassy, 102608927**Goals / deliverables:**

The goal of this spike task is demonstrate my ability to create a foundational message based interaction system for games where it allows entities such as players to interact with other entities such as a chest for example and communicate using messages while enabling actions to change states of entities.

Besides this report, what else was created?

- Code files in see COS30031-2023-102608927\19 - Spike - Messaging_ Annoucements and Blackboards\MessageSystem\MessageSystem

Technologies, Tools, and Resources used:

List of information needed by someone trying to reproduce this work.

- Visual Studio 2022 Community Edition
- GitHub
- The use of C++ Documentation - <https://en.cppreference.com>

Tasks undertaken:

- Creating a New Project using Visual Studio 2022
- Creating a message structure to encapsulate communication.
- Creating a message dispatcher class to handle registration listeners.
- Creating a base entity class to create other child entity classes.
- Compile and test code.

What we found out:**Implemented Message Class**

I created the message class to handle sending messages between entities where they carry details such as the sender, the message type and the associated data which is not a complex method but ensures that the functionality works when encapsulating the basic unit of communication.

```
Message::Message(const std::string& from, const std::string& to, const std::string& kind, const std::string& data)
: from(from), to(to), kind(kind), data(data) {}
```

Implemented Message Dispatcher Class

The Message Dispatcher Class basically manages the entire messaging system as its main role is to handle the messages as it ensures that messages are routed to the appropriate registered entities and is designed to be modular and ensures that the game entities are decoupled from each other so that is scalable and reusable.

```
void MessageDispatcher::registerListener(const std::string& entityId, std::function<void(const Message&)> listener) {
    listeners[entityId] = listener;
}

void MessageDispatcher::sendMessage(const Message& message) {
    if (listeners.count(message.to) > 0) {
        listeners[message.to](message);
    }
}
```

Implemented Entity Class

I created the entity base class so that it can serve as parent class to inherit from to create game entities such as player and chest so that we would receive and send messages when entities interact with each other.

```
Entity::Entity(const std::string& id, MessageDispatcher* dispatcher)
    : id(id), dispatcher(dispatcher) {}

Entity::~Entity() = default;

void Entity::sendMessage(const Message& message) {
    dispatcher->sendMessage(message);
}

std::string Entity::getId() const {
    return id;
}
```

Implementing Child Entities

Therefore, using the base entity class we can create two entities called player and chest to simulate a game world interaction where they can exchange messages to simulate some action such as player opens a chest and receives an item.

```
if (message.kind == "open") {
    bool isLocked = false; //I did this just to simply, if player is near a door he can go out
    if (isLocked) {
        std::cout << "Player" << id << "opened a chest" << message.to << std::endl;
    }
    else {
        std::cout << "Player" << id << "cant open chest because it is locked" << message.to << std::endl;
    }
}

else if (message.kind == "itemReceived") {
    std::cout << id << " received item: " << message.data << std::endl;
    // if I wanted to make it complicated I would have created an inventory system and add an item to the player's inventory.
}

else {
    std::cout << id << " received an unknown message type." << std::endl;
}
```

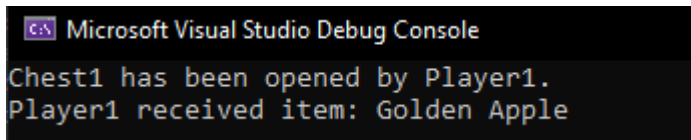
```
if (message.kind == "open") {
    bool isLocked = false; // just a placeholder for chest locked state
    std::string item = "Golden Apple"; // just a placeholder for the item inside the chest

    // Checks if the chest is locked
    if (!isLocked) {
        std::cout << getId() << " has been opened by " << message.from << "." << std::endl;

        // If chest is unlocked, it will send an "itemReceived" message back to the sender (player)
        Message itemMessage(getId(), message.from, "itemReceived", item);
        sendMessage(itemMessage);
    }
    else {
        // If the chest is locked, it will send a message back or handle it otherwise.
        std::cout << getId() << " is locked and cannot be opened by " << message.from << "." << std::endl;
    }
}
```

Finally, we can bring it together in the main loop to initialize the entities and message dispatcher to demonstrate the system where it simulates the example of player opening the chest and receiving an item that was hardcoded.

```
MessageDispatcher dispatcher;  
Player player("Player1", &dispatcher);  
Chest chest("Chest1", &dispatcher);  
  
// Registers player and chest with the dispatcher  
dispatcher.registerListener(player.getId(), [&player](const Message& message) {  
    player.receiveMessage(message);  
});  
  
dispatcher.registerListener(chest.getId(), [&chest](const Message& message) {  
    chest.receiveMessage(message);  
});  
  
// created an exmple to simulate player sending an "open" message to the chest  
Message openChestMessage(player.getId(), chest.getId(), "open", "");  
player.sendMessage(openChestMessage);
```



Microsoft Visual Studio Debug Console

```
Chest1 has been opened by Player1.  
Player1 received item: Golden Apple
```

Open issues/risks

Some issue this implementation faces would be the current synchronous messaging system that can sometimes not scale well when we have multiple entities or any complex interactions.

Recommendations

To look into asynchronous messaging systems that improve scalability and also some extra implementations such as to enhance the robustness of message handling for more complex interactions.